

Appendice integrativa

Admin SuperUser e motore Ollama

UniTorV E-Vote

Aggiornamento alla relazione Cyber_By_Design.pdf

Questa appendice è pensata per essere concatenata alla relazione `Cyber_By_Design.pdf`. Integra la documentazione tecnica con due evoluzioni introdotte nel progetto: l'aggiunta dell'utente Admin, indicato come **SuperUser**, distinto dal profilo **CEO**, e l'integrazione del motore Ollama per l'analisi assistita dei log applicativi tramite le route definite in `Server/routes/ollama_route.js`.

22 Aggiornamento funzionale e architetturale

L'obiettivo dell'aggiornamento è rafforzare la separazione dei compiti. Il **CEO** mantiene le funzioni operative e decisionali sulla piattaforma di voto, mentre l'**ADMIN** svolge attività di monitoraggio e consultazione dei log.

22.1 Nuovo attore: Admin SuperUser

Il sistema introduce un profilo amministrativo separato dal **CEO**. Nel codice applicativo il ruolo è rappresentato dalla classe **ADMIN**, mentre nella relazione può essere descritto come utente Admin o **SuperUser**.

Campo	Valore
File	Server/user_script/databaseChiaro.csv
id_wallet	31
Nome/Cognome	Admin Admin
Email	admin.admin@example.com
Classe applicativa	ADMIN
Quota	0.00

Per minimizzazione delle informazioni sensibili, le credenziali di accesso non devono essere riportate nella relazione finale. La loro gestione resta confinata al dataset di test o alla procedura di provisioning prevista per l'ambiente di esecuzione.

22.2 Separazione tra Admin e CEO

L'aggiunta dell'Admin non estende i privilegi del **CEO** e non trasforma l'Admin in un **CEO** tecnico. I due ruoli hanno responsabilità distinte.

Ruolo	Responsabilità principali	Pagine/API previste
CEO	Creazione riunioni, gestione utenti, gestione votazioni, consultazione risultati e verbali	Dashboard CEO, rotte protette da <code>isCeo</code>
ADMIN	Monitoraggio, visualizzazione log e analisi assistita degli eventi	<code>logVisual.html</code> , <code>POST /api/v1/logs/analyze</code>
membro	Partecipazione alle riunioni e alle votazioni autorizzate	Dashboard membro e sala di voto

Il ruolo **ADMIN** è progettato secondo il principio del least privilege: può osservare gli eventi registrati dal sistema, ma non può creare riunioni, aggiungere utenti, gestire votazioni o alterare gli esiti.

22.3 Flusso di autenticazione e redirect dell'Admin

Il login resta centralizzato nella rotta `POST /api/v1/loginCheck`, definita in `Server/routes/login_route.js`. Dopo la validazione della password, il backend genera un JWT con i campi `id_wallet`, `classe` ed `email`. Il token viene salvato in un cookie `httpOnly`; il frontend riceve i dati utente e determina la destinazione iniziale.

In `Public/Sources/js/login.js` è stato aggiunto il controllo del ruolo Admin:

```
if (ruoloRaw.includes("admin")) {
    window.location.href = "logVisual.html";
} else if (ruoloRaw.includes("ceo")) {
    window.location.href = "ceo_dashboard.html";
} else {
    window.location.href = "member_dashboard.html";
}
```

In questo modo l'utente con classe **ADMIN** viene indirizzato automaticamente alla pagina di monitoraggio dei log dopo il login.

Nota di allineamento terminologico: nella richiesta funzionale la pagina è indicata come `VisualLog.html`; nel repository il file effettivo è `Public/logVisual.html`.

22.4 Protezione della pagina di visualizzazione log

La protezione lato server delle pagine statiche è gestita da `Server/middleware/path_middle.js`. Il middleware introduce l'elenco delle pagine riservate all'Admin:

```
const adminPages = ["/logvisual.html"];
```

Quando una richiesta punta alla pagina `logVisual.html`, il middleware verifica la presenza del cookie JWT, valida il token, normalizza il campo `classe`, consente l'accesso solo se il ruolo è **ADMIN**, registra un alert in caso di accesso non autorizzato e reindirizza gli utenti non autorizzati alla dashboard coerente con il loro ruolo.

Il controllo è separato da quello CEO:

```
const isCEO = ruolo === "CEO";
const isAdmin = ruolo === "ADMIN";
```

Questo garantisce che l'accesso alla console dei log non dipenda dal ruolo CEO, ma da un profilo amministrativo dedicato.

22.5 Funzionalità della pagina logVisual.html

La pagina `Public/logVisual.html`, insieme allo script `Public/Sources/js/logVisual.js`, implementa una dashboard di monitoraggio con recupero dello storico dei log, parsing del formato applicativo, visualizzazione tabellare, filtro per `Alert` o `Signal`, ordinamento temporale, refresh manuale, selezione multipla degli eventi e invio dei log selezionati a Ollama per analisi correlata.

Il formato log atteso è:

```
[[<timestamp>][<ip-sorgente>][<ip-destinazione>]
  [<method> <route>][<codice HTTP>][<commento>]]
```

La tabella mantiene anche la stringa grezza originale del log, salvata nel dataset della checkbox, così da inviarla al motore Ollama senza perdere contesto.

22.6 Recupero dei log dalla blockchain

La rotta `Server/routes/logger_route.js` espone `GET /api/v1/logs/history`. La rotta recupera lo storico dalla classe blockchain `Logger`, usando le chiavi `Alert` e `Signal`.

```
getHistoryKV("Logger", "Alert")
getHistoryKV("Logger", "Signal")
```

Il backend restituisce al frontend un array normalizzato:

```
[
  {
    "type": "Alert",
    "raw": "[[...]]"
  }
]
```

Questa impostazione conserva la tracciabilità degli eventi su blockchain e permette alla dashboard Admin di presentare una vista leggibile senza alterare il dato originale.

23 Motore Ollama per analisi dei log

Il progetto integra Ollama come motore locale di analisi LLM per supportare l'Admin nella lettura degli eventi di sicurezza. La route è registrata in `Server/server.js`:

```
const ollamaRoute = require("../routes/ollama_route.js");
app.use("/api/v1/", ollamaRoute);
```

La dipendenza è dichiarata in `Server/package.json` con versione `^0.6.3`.

23.1 Endpoint di analisi batch

Il file `Server/routes/ollama_route.js` definisce l'endpoint `POST /api/v1/logs/analyze`. La rotta è protetta dai middleware `verifyToken` e `isAdmin`:

```
router.post("/logs/analyze", verifyToken, isAdmin, async (req, res) => {  
    ...  
});
```

Questa scelta vincola l'analisi AI al solo profilo `ADMIN`: un CEO o un membro non possono invocare direttamente l'endpoint se non possiedono un JWT con classe `ADMIN`.

Il payload atteso contiene un array non vuoto `logLines`. In caso di assenza o formato errato, la rotta restituisce `400 Bad Request`.

```
{  
  "logLines": [  
    "[[2026-06-01T10:00:00.000Z] [10.0.0.1] [10.0.0.2]  
    [POST /api/v1/loginCheck] [401] [Password errata]]"  
  ]  
}
```

23.2 Prompt di sistema e baseline di sicurezza

Ollama viene guidato tramite la costante `BATCH_BASELINE_SYSTEM_PROMPT`, che imposta il modello come analista senior di cybersecurity del sistema `UniTorV E-Vote`.

Il prompt contiene formato dei log, classificazione `SIGNAL/ALERT`, baseline dei percorsi di rete secondo IEC 62443, baseline dei flussi API per ruolo, anomalie applicative note, mappatura STRIDE e schema JSON obbligatorio per la risposta.

Evento	Interpretazione
403 su <code>POST /api/v1/add-vote</code>	Possibile double voting rilevato dal middleware di controllo voto
404 su <code>GET /api/v1/visualize-vote</code>	Tentativo prematuro di visualizzazione dell'esito
401 su <code>POST /api/v1/loginCheck</code>	Password errata, possibile brute force o credential stuffing se ripetuto
500 su <code>POST /api/v1/uploadVerbale</code>	Errore caricamento verbale o possibile Denial of Service

23.3 Chiamata al modello Ollama

La rotta aggrega i log selezionati in un unico blocco:

```
const logsPayload = logLines  
  .map((line, idx) => `[Log ${idx + 1}] ${line}`)  
  .join("\n");
```

Successivamente invoca il modello llama3.2 tramite `ollama.chat`, con temperatura 0.1. La temperatura bassa riduce la variabilità della risposta e favorisce una classificazione più stabile degli eventi.

```
const response = await ollama.chat({
  model: "llama3.2",
  messages: [
    { role: "system", content: BATCH_BASELINE_SYSTEM_PROMPT },
    {
      role: "user",
      content: `Ecco la lista di log da analizzare insieme:\n\n${logsPayload}`,
    },
  ],
  options: { temperature: 0.1 },
});
```

23.4 Output strutturato dell'analisi

Il backend richiede a Ollama di restituire esclusivamente un oggetto JSON con sintesi correlata, livello di rischio complessivo e dettaglio dei singoli log.

```
{
  "mini_storico_globale": "Sintesi correlata degli eventi",
  "livello_rischio_complessivo": "BASSO|MEDIO|ALTO|CRITICO",
  "analisi_dettagliata": [
    {
      "indice": 1,
      "tipo_rilevato": "SIGNAL|ALERT",
      "route": "endpoint rilevato",
      "codice_http": 401,
      "analisi_sicurezza": "Spiegazione rispetto a baseline, RBAC e STRIDE",
      "raccomandazione": "Mitigazione suggerita"
    }
  ]
}
```

Se il contenuto generato dal modello è JSON valido, la rotta restituisce direttamente l'oggetto al frontend. Se il parsing fallisce, il backend mantiene la risposta grezza in `raw_analysis` e aggiunge un campo `warning`, evitando il blocco dell'interfaccia.

23.5 Visualizzazione del report AI

Nel frontend, la funzione `analyzeSelectedLogs()` raccoglie le checkbox selezionate, estrae le stringhe log originali, invia una richiesta POST `/api/v1/logs/analyze`, riceve la risposta JSON e apre un modal di report tramite `showAnalysisModal()`.

Il modal presenta mini storico e correlazione degli eventi, livello di rischio complessivo, dettaglio dei singoli log, classificazione `SIGNAL/ALERT`, spiegazione `STRIDE` e baseline, raccomandazione di mitigazione.

24 Aggiornamento dei casi d'uso

24.1 UC-05 - Accesso Admin alla dashboard log

Attore primario: Admin SuperUser.

Precondizioni: l'utente è registrato con classe ADMIN e possiede credenziali valide.

Flusso principale: l'Admin accede alla pagina di login, inserisce identificativo e password, il backend valida le credenziali e genera il JWT, il frontend rileva il ruolo ADMIN, l'utente viene reindirizzato a `logVisual.html`, il middleware consente l'accesso alla pagina solo se il token contiene classe ADMIN, quindi la pagina carica e mostra gli eventi **Alert** e **Signal**.

Postcondizione: l'Admin visualizza lo storico dei log senza poter modificare riunioni, votazioni o utenti.

24.2 UC-06 - Analisi assistita dei log tramite Ollama

Attore primario: Admin SuperUser.

Precondizioni: l'Admin è autenticato e si trova nella pagina `logVisual.html`.

Flusso principale: l'Admin seleziona uno o più log dalla tabella, il frontend invia i log selezionati a `POST /api/v1/logs/analyze`, il backend verifica JWT e ruolo ADMIN, la rotta costruisce il prompt batch, Ollama analizza i log rispetto a baseline IEC 62443, RBAC e STRIDE, il backend valida o normalizza l'output, infine il frontend mostra un report strutturato.

Postcondizione: l'Admin ottiene una sintesi di sicurezza consultiva e non vincolante, utile per triage e investigazione.

25 Impatto sul modello STRIDE

Categoria STRIDE	Impatto dell'aggiornamento
Spoofing	Il JWT limita l'accesso alla dashboard Admin agli utenti autenticati con classe ADMIN.
Tampering	L'Admin osserva i log ma non dispone di funzioni per modificarli o alterare votazioni.
Repudiation	I tentativi di accesso non autorizzati vengono registrati come Alert .
Information Disclosure	I log sono dati sensibili e richiedono protezione API e UI tramite RBAC.
Denial of Service	L'endpoint Ollama può essere gravoso: sono consigliati limiti di dimensione, rate limiting e timeout.
Elevation of Privilege	La separazione <code>isCeo/isAdmin</code> riduce il rischio che il CEO o un membro usino funzioni di monitoraggio riservate.

26 Requisiti operativi e raccomandazioni

Per l'esercizio corretto del motore Ollama, il servizio deve essere installato e raggiungibile dall'ambiente Node.js, il modello `llama3.2` deve essere disponibile localmente, il server deve avere risorse sufficienti per l'inferenza e l'endpoint deve gestire errori di connessione, indisponibilità del modello e output non JSON.

L'analisi AI deve rimanere consultiva e non deve eseguire azioni automatiche sul sistema.

27 File modificati o coinvolti

Area	File	Ruolo
Login frontend	<code>Public/Sources/js/login.js</code>	Redirect Admin verso <code>logVisual.html</code>
Pagina Admin	<code>Public/logVisual.html</code>	Dashboard visualizzazione log
Script pagina Admin	<code>Public/Sources/js/logVisual.js</code>	Recupero, filtro, selezione e analisi log
Protezione pagine	<code>Server/middleware/path_middle.js</code>	Controllo accesso a <code>logVisual.html</code>
Autorizzazione API	<code>Server/middleware/auth_middle.js</code>	Middleware <code>isAdmin</code> separato da <code>isCeo</code>
Recupero log	<code>Server/routes/logger_route.js</code>	Endpoint GET <code>/api/v1/logs/history</code>
Motore AI	<code>Server/routes/ollama_route.js</code>	Endpoint POST <code>/api/v1/logs/analyze</code>
Registrazione route	<code>Server/server.js</code>	Mount di <code>loggerRoute</code> e <code>ollamaRoute</code>
Dipendenze	<code>Server/package.json</code>	Dipendenza <code>ollama</code>

28 Conclusione

L'introduzione dell'Admin SuperUser e dell'integrazione Ollama estende UniTorV E-Vote verso un modello di monitoraggio più maturo. La piattaforma separa le responsabilità operative del CEO dalle attività di audit dell'Admin e aggiunge un livello di analisi assistita capace di correlare eventi, classificare anomalie e proporre raccomandazioni secondo baseline IEC 62443, RBAC e STRIDE.

L'aggiornamento è coerente con l'approccio cyber by design: riduce l'accumulo di privilegi, migliora l'osservabilità, valorizza i log salvati su blockchain e supporta l'investigazione degli eventi senza delegare decisioni critiche al modello LLM.